

```
import pandas as pd
import numpy as np
import re
```

```
Sales = pd.read_csv("/content/Sales.csv")
Sales.shape
```

(9950, 12)

```
!pip install ydata-profiling --quiet
```

```
from ydata_profiling import ProfileReport
```

```
profile = ProfileReport(Sales, title="Sales Report")
profile.to_file("Sales_profile.html")
profile
```

Summarize dataset: 100%71/71 [00:16<00:00, 6.07it/s, Completed]

0%| | 0/12 [00:00<?, ?it/s]

25%|██| 3/12 [00:00<00:00, 24.97it/s]

50%|██████| 6/12 [00:00<00:00, 20.40it/s]

75%|██████████| 9/12 [00:00<00:00, 16.65it/s]

100%|██████████████| 12/12 [00:00<00:00, 16.51it/s]

Generate report structure: 100%1/1 [00:04<00:00, 4.17s/it]

Render HTML: 100%1/1 [00:00<00:00, 2.53it/s]

/usr/local/lib/python3.13/dist-packages/ydata_profiling/profile_report.py:386: UserWarning: Extension not supported. For warnings.warn()

Export report to file: 100%1/1 [00:00<00:00, 33.14it/s]

Sales Report

Overview

OverviewAlerts 6Reproduction

Dataset statistics

Number of variables	12
Number of observations	9950
Missing cells	17389
Missing cells (%)	14.6%
Duplicate rows	50
Duplicate rows (%)	0.5%
Total size in memory	932.9 KiB
Average record size in memory	96.0 B

Variable types

Numeric	7
Categorical	4
DateTime	1

Variables

Select Columns

sale_id

```
def extract_promo(note):
    if pd.isna(note) or note == " ":
```

```

return pd.Series({"promo_code": None, "discount_reason": None})

m = re.search(r"Promo code (\w+) applied at checkout", note)
if m:
    return pd.Series({"promo_code": m.group(1), "discount_reason": "promo code"})

if "competitor" in note.lower():
    return pd.Series({"promo_code": None, "discount_reason": "price_match"})

if "loyalty" in note.lower():
    return pd.Series({"promo_code": None, "discount_reason": "loyalty_tier"})

return pd.Series({"promo_code": None, "discount_reason": "other"})

extracted = Sales["notes"].apply(extract_promo)
Sales = pd.concat([Sales, extracted], axis=1)
Sales

```

	sale_id	store_id	employee_id	product_id	customer_id	quantity	sale_date	unit_price	discount_pct	payment_method
0	2	11	44	4308	1300	3	6/10/2025	56.69	29.6	Debit Card
1	3	11	63	2359	586	5	9/12/2025	104.55	33.7	Credit Card
2	4	10	33	44	1381	2	18/08/2025	38.39	25.6	Credit Card
3	5	12	60	3991	146	5	12/12/2024	21.74	4.6	Debit Card
4	6	10	16	1274	158	5	2/08/2025	102.58	32.3	Cash
...	...	...	...	...	...	...	...	...	...	...
9945	9892	6	49	1797	146	5	7/06/2025	249.89	7.8	credit card
9946	9893	5	92	3021	1189	4	27/01/2025	229.34	7.1	debit card
9947	9896	8	116	993	1181	2	25/10/2024	238.65	9.3	online
9948	9899	7	5	426	298	3	21/06/2025	18.37	6.1	Cash
9949	9900	6	107	4070	202	5	26/11/2025	66.02	4.8	CASH

9950 rows × 14 columns

Next steps: [Generate code with Sales](#) [New interactive sheet](#)

```
Sales["discount_reason"].value_counts(dropna=False)
```

```

count
discount_reason
None      8294
promo code    855
price_match   461
loyalty_tier  340

dtype: int64

```

```
Sales['notes'].unique()
```

```

array([nan, 'Promo code LOYALTY20 applied at checkout',
       'Promo code WELCOME15 applied at checkout',
       'Price matched to competitor flyer',
       'Promo code SAVE10 applied at checkout',
       'Loyalty tier discount applied',
       'Promo code SEASON25 applied at checkout'], dtype=object)

```

```
Sales[Sales['discount_reason'] == 'other']
```

```
sale_id store_id employee_id product_id customer_id quantity sale_date unit_price discount_pct payment_method not
```

Feature Engineering

```
products = pd.read_csv("/content/products_catalog.csv")
```

```
Sales = Sales.merge(products[['product_id', 'cost_price']])
```

Sales

	sale_id	store_id	employee_id	product_id	customer_id	quantity	sale_date	unit_price	discount_pct	payment_method
0	2	11	44	4308	1300	3	6/10/2025	56.69	29.6	Debit Card
1	3	11	63	2359	586	5	9/12/2025	104.55	33.7	Credit Card
2	4	10	33	44	1381	2	18/08/2025	38.39	25.6	Credit Card
3	5	12	60	3991	146	5	12/12/2024	21.74	4.6	Debit Card
4	6	10	16	1274	158	5	2/08/2025	102.58	32.3	Cash
...	...	...	...	...	...	...	...	...	...	...
9945	9892	6	49	1797	146	5	7/06/2025	249.89	7.8	credit card
9946	9893	5	92	3021	1189	4	27/01/2025	229.34	7.1	debit card
9947	9896	8	116	993	1181	2	25/10/2024	238.65	9.3	online
9948	9899	7	5	426	298	3	21/06/2025	18.37	6.1	Cash
9949	9900	6	107	4070	202	5	26/11/2025	66.02	4.8	CASH

9950 rows × 15 columns

Next steps: [Generate code with Sales](#) [New interactive sheet](#)

```
Sales['total_amount'] = ( Sales['quantity'] * Sales['unit_price'] * (1-Sales['discount_pct']/100))
```

```
Sales['cost_amount'] = (Sales['quantity'] * Sales['cost_price']).round(2) #1.2222222222 -> 1.22
```

```
Sales['profit'] = (Sales['total_amount'] - Sales['cost_amount']).round(2)
```

```
Sales['margin'] = (Sales['profit'] / Sales['total_amount']*100).round(2)
```

Sales.head(10)

	sale_id	store_id	employee_id	product_id	customer_id	quantity	sale_date	unit_price	discount_pct	payment_method
0	2	11	44	4308	1300	3	6/10/2025	56.69	29.6	Debit Card
1	3	11	63	2359	586	5	9/12/2025	104.55	33.7	Credit Card
2	4	10	33	44	1381	2	18/08/2025	38.39	25.6	Credit Card
3	5	12	60	3991	146	5	12/12/2024	21.74	4.6	Debit Card
4	6	10	16	1274	158	5	2/08/2025	102.58	32.3	Cash
5	15	10	101	3653	80	3	27/01/2025	185.64	25.4	Credit Card
6	18	11	63	4795	795	5	1/08/2025	213.27	32.4	Cash
7	24	9	61	3820	1362	2	10/01/2025	233.40	7.4	Credit Card
8	26	11	97	2982	1362	1	4/09/2025	164.75	29.5	Credit Card
9	29	11	64	4913	120	1	25/01/2025	96.45	26.6	Cash

Next steps: [Generate code with Sales](#) [New interactive sheet](#)

```
customers = pd.read_csv("/content/customers_master.csv")
```

```
# important for RepeatCustomerKPI
def extract_loyalty(note):
    if pd.isna(note) or note == "":
        return pd.Series({"loyalty_tier": None, "loyalty_since": None})

    m = re.search(r"(Silver|Gold|Platinum) member - loyalty since (\d{4})", note)
    if m:
        return pd.Series({"loyalty_tier": m.group(1), "loyalty_since": int(m.group(2))})
    return pd.Series({"loyalty_tier": None, "loyalty_since": None})

loyalty = customers["notes"].apply(extract_loyalty)
loyalty
customers = pd.concat([customers, loyalty], axis=1)
```

```
customers["loyalty_tier"].value_counts(dropna=False)
```

	count
loyalty_tier	
None	1099
Silver	158
Platinum	154
Gold	144

dtype: int64

```
customers["phone_clean"] = customers["phone"].astype(str).str.replace(r"\D", "", regex=True)
```

```
customers['city_clean'] = customers['city'].astype(str).str.lower().str.strip()
```

```
duplicate_groups = [
    {
        "phone": phone,
        "customer_ids": group["customer_id"].tolist()
    }
    for phone, group in customers.groupby("phone_clean")
    if len(group) > 1
]

duplicate_groups
```

```
[{'phone': '0120312738', 'customer_ids': [1529, 1368]},
 {'phone': '0346718657', 'customer_ids': [1523, 1262]},
 {'phone': '0903958146', 'customer_ids': [227, 1539]},
 {'phone': '1070983552', 'customer_ids': [1288, 1551]},
 {'phone': '1232566042', 'customer_ids': [11, 1507]},
 {'phone': '1387506072', 'customer_ids': [1511, 490]},
 {'phone': '1403780254', 'customer_ids': [201, 1545]},
 {'phone': '1918453122', 'customer_ids': [182, 1502]},
 {'phone': '2082302039', 'customer_ids': [92, 1542]},
 {'phone': '2364021542', 'customer_ids': [1505, 1308]},
 {'phone': '2519836473', 'customer_ids': [1532, 43]},
 {'phone': '2719200342', 'customer_ids': [1536, 1363]},
 {'phone': '2861539636', 'customer_ids': [1543, 872]},
 {'phone': '2930495031', 'customer_ids': [1541, 1492]},
 {'phone': '3070107286', 'customer_ids': [1518, 1453]},
 {'phone': '3100486159', 'customer_ids': [85, 1517]},
 {'phone': '3122820605', 'customer_ids': [1525, 426]},
 {'phone': '3350119341', 'customer_ids': [136, 1510]},
 {'phone': '3914256511', 'customer_ids': [732, 1554]},
 {'phone': '3991931598', 'customer_ids': [232, 1548]},
 {'phone': '4453111141', 'customer_ids': [1527, 1327]},
 {'phone': '4554728484', 'customer_ids': [837, 1524]},
 {'phone': '5156393203', 'customer_ids': [806, 1552]},
 {'phone': '5213537876', 'customer_ids': [1187, 1533]},
 {'phone': '5560399891', 'customer_ids': [1508, 1346]},
 {'phone': '6091391101', 'customer_ids': [1509, 220]},
 {'phone': '6296743145', 'customer_ids': [1512, 1147]},
 {'phone': '6474935237', 'customer_ids': [704, 1503]},
 {'phone': '6480297227', 'customer_ids': [1226, 1504]},
 {'phone': '6592169945', 'customer_ids': [1513, 446]},
 {'phone': '6619577471', 'customer_ids': [731, 1553]},
 {'phone': '6819081876', 'customer_ids': [1534, 411]},
```

```
{'phone': '7110878738', 'customer_ids': [1546, 366]},  
{'phone': '7508920694', 'customer_ids': [1351, 1528]},  
{'phone': '7624741910', 'customer_ids': [1535, 1138]},  
{'phone': '7630544575', 'customer_ids': [1506, 518]},  
{'phone': '7821678948', 'customer_ids': [1501, 86]},  
{'phone': '8408967147', 'customer_ids': [29, 1515]},  
{'phone': '8982484003', 'customer_ids': [1540, 280]},  
{'phone': '9091405791', 'customer_ids': [1550, 742]},  
{'phone': '940211340', 'customer_ids': [1544, 843]},  
{'phone': '9507472950', 'customer_ids': [159, 1514]}
```